

Full Technical Architecture, Code Walkthrough & Interview Defense Kit

A Master Blueprint for Junior & Fresher Software Engineers to Ace Senior Technical Interviews

Target Role: Software Development Engineer / Backend Engineer	Tech Stack: Spring Boot 3, FastAPI, Next.js 14, PostgreSQL
Domain: Fintech & Real-Time Cyber Threat Intelligence	Repository: github.com/Piyush-711/fraudShield
Author / Candidate: Piyush	Architecture: Hybrid Event-Driven Microservices

1. Executive Summary & Problem Statement

The Real-World Financial Crisis: Digital payment volumes have exploded, bringing sophisticated fraud vectors including automated card testing, distributed credential stuffing, synthetic account takeover, and rapid geographic velocity anomalies. Traditional banking infrastructure relied on rigid, static rules (e.g. *'Flag transactions over \$5,000'*). Fraudsters easily circumvent static rules by initiating micro-transactions under thresholds across rotating IP subnets. Conversely, aggressive machine learning models frequently trigger false positives, declining genuine users and hurting merchant revenue.

FraudShield's Solution: An enterprise, sub-200ms hybrid risk adjudication platform combining supervised machine learning (XGBoost), unsupervised anomaly detection (Isolation Forest), and an in-memory rule engine with dynamic threshold management. It introduces a 3-tier adjudication funnel:

Funnel Tier	Score Range	Action Taken	System SLA
Auto-Approved	0 to 29	Synchronous approval returned to bank gateway	< 150 ms
Manual Review	30 to 69	Enqueued into Analyst portal with explainability factors	Async Triage
Auto-Rejected	70 to 100	Hard block executed; Security alert triggered	< 150 ms

2. Complete Tech Stack & Architectural Justifications

When an interviewer asks 'Why did you choose this technology?', provide these battle-tested, concise responses:

Component	Technology	Interview Defense Answer (1-2 Sentences)
Backend Framework	Java 17 / Spring Boot 3.2	'Spring Boot provides enterprise-grade thread pooling, dependency injection, and declarative transaction management critical for high-throughput banking systems.'
Machine Learning	Python 3.11 / FastAPI	'FastAPI provides native ASGI async event loops with Pydantic validation, offering near-C++ network throughput while accessing Python's mature ML ecosystem.'
ML Algorithms	XGBoost + Isolation Forest	'XGBoost provides superior accuracy on labeled tabular data, while Isolation Forest catches novel zero-day anomalies that didn't exist in training callsets.'
Database	PostgreSQL + Flyway	'PostgreSQL guarantees strict ACID compliance and row-level locking for financial ledgers; Flyway ensures version-controlled, repeatable schema migrations.'
Cache & Rate Limit	Redis (Lettuce Driver)	'Redis delivers sub-millisecond atomic key-value increments for sliding-window rate limiters and user transaction velocity tracking outside JVM heap memory.'
Frontend Shell	Next.js 14 (App Router)	'Next.js 14 App Router gives us Server-Side Rendering (SSR) for instant first paint, combined with strict TypeScript contracts matching backend DTO schemas.'
Security & Auth	Spring Security + JWT	'Stateless JWT with HMAC-SHA256 signatures eliminates server session clustering bottlenecks while enforcing fine-grained Role-Based Access Control (RBAC).'

3. System Architecture & End-to-End Workflow

The Life of a Transaction Request (Step-by-Step Execution):

- Gateway Ingestion:** Core banking upstream calls POST /api/v1/transactions/evaluate with an X-API-KEY header and JSON payload containing user ID, card hash, merchant name, amount, location, and device details.
- Rate Limiting Check:** RateLimitService checks the client IP / API key against a 60-second sliding-window counter in Redis (or ConcurrentHashMap fallback). If rate > 5,000 req/min, it rejects immediately with HTTP 429.
- Static Rule Engine:** RuleEngine.java evaluates active business rules from FraudRuleRepository (e.g. user hourly velocity, blacklisted merchant categories, high-risk destination countries).

- 4. Parallel ML Inference:** TransactionService.java calls the FastAPI microservice via connection-pooled RestTemplate (500ms connect, 1500ms read timeout). The ML service runs XGBoost + Isolation Forest, outputting a risk score (0-100), confidence percentage, and top explainability weights.
- 5. Hybrid Decision Synthesis:** The final risk score is computed: finalScore = max(mlScore, rulePenalty). The score is evaluated against dynamic thresholds from SystemConfig: APPROVED (<30), MANUAL_REVIEW (30-70), or REJECTED (>70).
- 6. ACID Persistence & Audit Log:** Inside a @Transactional boundary, the transaction, explainability factors JSON, and initial audit log entries are saved to PostgreSQL. If high risk, a SystemAlert is created.
- 7. Synchronous Gateway Response:** The client receives HTTP 200 with decision and transaction ID in under 150 milliseconds.
- 8. Human Adjudication (If Flagged):** An analyst views the transaction in Next.js 14, inspects SHAP factor bars, and submits an approval or rejection with justification notes via POST /api/v1/transactions/{id}/review.

4. Core Features & Codebase File Map

Feature Module	Key Source Files	Core Responsibilities
Transaction Ingestion & Orchestration	TransactionController.java TransactionService.java	Validates payloads, coordinates parallel calls to ML service, evaluates business rules, commits DB transactions.
Dynamic Rule Engine	RuleEngine.java FraudDetectionRule.java	Executes velocity, amount, and geolocation heuristics loaded from PostgreSQL without requiring app restarts.
ML Inference & Explainability	app/main.py services/ml_service.py	Vectorized feature extraction, XGBoost classification, Isolation Forest anomaly scoring, and SHAP factor decomposition.
Stateless Authentication	JwtService.java SecurityConfig.java	HMAC-SHA256 JWT generation/verification, SecurityFilterChain, and real-time active status revocation check.
Rate Limiting	RateLimitService.java	Sliding-window rate limiting with Redis implementation and local thread-safe ConcurrentHashMap fallback.
Human Adjudication Portal	app/dashboard/* app/admin/*	Next.js 14 App Router, live sparklines, explainability detail view, review adjudication modal, zero raw emojis.

5. Database Schema & Data Modeling

Table Name	Key Columns & Types	Constraints & Indexing Rationale
transactions	transaction_id (PK, VARCHAR) user_id (VARCHAR) amount (DECIMAL 15,2) fraud_score (INT) transaction_status (VARCHAR) factors_json (TEXT)	Primary key indexed for O(1) lookups. Composite index on (user_id, created_at) for sub-10ms user velocity checks. Index on created_at for dashboard aggregation.
app_users	id (PK, BIGINT) email (UK, VARCHAR) password_hash (VARCHAR) role (VARCHAR) is_active (BOOLEAN)	Unique constraint on email. Checked by JwtAuthenticationFilter on every authenticated request to immediately block deactivated users.
system_alerts	id (PK, BIGINT) alert_type (VARCHAR) severity (VARCHAR) status (VARCHAR) transaction_id (FK)	Foreign key to transactions. Filtered by status='ACTIVE' for the real-time operational incident feed.
system_config	config_key (UK, VARCHAR) config_value (VARCHAR) data_type (VARCHAR)	Stores threshold boundaries (auto_approval_threshold, etc.). Loaded into memory cache to allow dynamic threshold updates without service redeployment.
audit_logs	id (PK, BIGINT) action_type (VARCHAR) actor_id (VARCHAR) old_value, new_value (VARCHAR)	Immutable compliance log tracking state transitions (e.g. PENDING -> APPROVED) and human overrides for audit verification.

6. Challenges, Edge Cases & Decisions Handled

Lombok @Builder Default Field Erasure:

When Lombok's @Builder is applied to an entity, fields with inline default values (like private boolean isActive = true;) get erased and initialized to Java defaults (false). In our project, new users were being created as deactivated, locking them out immediately. **Resolution:** Annotated all initialized fields with @Builder.Default and wrote automated test coverage in AdminServiceTest.

Socket Starvation & TIME_WAIT Leak Under Load:

Using standard unpooled HTTP clients resulted in socket starvation under 500 TPS because every HTTP call opened a new TCP handshake, quickly consuming ephemeral ports. **Resolution:** Configured a connection-pooled RestTemplate backed by Apache PoolingHttpClientConnectionManager with 200 max connections, a 500ms connect timeout, and a 1500ms read timeout in WebConfig.java.

Next.js Server-Side Rendering (SSR) Hydration Mismatch:

Rendering dynamic time (`new Date().toLocaleTimeString()`) in Next.js App Router caused hydration errors because the server-rendered HTML timestamp differed from the client browser timestamp 2 seconds later. **Resolution:** Implemented a mounted state guard with `suppressHydrationWarning` to guarantee identical initial DOM trees during hydration.

Concurrency at 10,000 Simultaneous Requests (What Breaks & How to Scale):

1. *HikariCP Database Pool Exhaustion:* Default 10 connections would block under high concurrency. **Scale:** Increase Hikari pool size to 50, add read replicas for reporting queries, and buffer ingest writes via partitioned Apache Kafka.
2. *In-Memory Rate Limiting:* Single-node `ConcurrentHashMap` scales linearly in heap size. **Scale:** Switch to distributed Redis cluster sliding-logs.
3. *ML CPU Saturation:* Python GIL can bottleneck on CPU. **Scale:** Export trained models to ONNX runtime or NVIDIA Triton server in C++ for GPU-accelerated 5ms inference.

7. Fresher Interview Defense Kit

The 90-Second Elevator Pitch (Memorize and Practice):

*"For my major project, I engineered **FraudShield**, a real-time financial fraud detection and adjudication platform designed for banks and payment gateways.*

The core problem we solved is that traditional rule-based systems are too rigid, while purely ML-based systems lack explainability and cause high false-positive rates. I designed a hybrid architecture: incoming transactions hit a Spring Boot microservice where they undergo sliding-window rate limiting. The transaction is then evaluated in parallel by a business rules engine and an asynchronous Python FastAPI machine learning service running XGBoost and Isolation Forest models.

The system computes a composite risk score from 0 to 100 in under 150 milliseconds. Scores below 30 are auto-approved, scores above 70 are blocked, and borderline transactions are routed to a human review queue. On the frontend, I built a responsive Next.js 14 dashboard using TypeScript and Tailwind CSS, where fraud analysts can inspect feature explainability factors, review transactions, and adjust risk thresholds dynamically.

Through this project, I gained deep hands-on experience with microservice communication, connection pooling, database migrations with Flyway, and securing endpoints using stateless JWT authentication."

Top 10 Senior Interview Questions & Crisp Model Answers:

Q1: Why separate the ML service into Python and the backend into Java?

Java and Spring Boot excel at multi-threaded transactional I/O, ACID persistence, and enterprise security. Python is the native environment for data science and ML libraries (XGBoost, Scikit-learn). Separating them into microservices allows scaling compute-heavy ML inference independently from I/O-heavy database workloads.

Q2: How do you guarantee the fraud decision takes less than 200ms?

We optimize every stage: 1) The HTTP client uses pre-warmed connection pooling with strict 500ms connect and 1500ms read timeouts; 2) The ML service keeps models loaded in memory; 3) Database lookups use composite indexes (transaction_id, user_id + created_at); 4) If the ML service times out, a circuit breaker falls back to the static rules engine to prevent customer wait times.

Q3: How does your JWT authentication handle deactivated accounts in real time?

Stateless JWTs normally remain valid until expiration. In FraudShield, our JwtAuthenticationFilter extracts the claims and performs a lightweight indexed check: user.getIsActive(). If an admin deactivates an account, access is rejected immediately on the very next request without needing token blacklists.

Q4: What is the difference between XGBoost and Isolation Forest in your pipeline?

XGBoost is a supervised ensemble model trained on labeled historical fraud to recognize known attack patterns (e.g., velocity spikes). Isolation Forest is an unsupervised tree model that isolates anomalies by randomly partitioning features; it does not require labels and catches novel zero-day attacks that have never been seen before.

Q5: What is Flyway, and why not use hibernate.hbm2ddl.auto = update?

Using ddl-auto=update in production risks silent table locks and data corruption during non-backward-compatible column changes. Flyway treats database migrations as versioned code (V1__init_schema.sql). Migrations execute sequentially, idempotently, and are logged in flyway_schema_history.

Q6: How does your sliding-window rate limiter work?

A fixed window resets at fixed clock intervals, permitting 2x traffic bursts at the boundary. A sliding window evaluates a rolling 60-second window. Incoming request timestamps are recorded in a thread-safe deque or Redis sorted set; timestamps older than 60 seconds are evicted, and requests exceeding the threshold receive HTTP 429.

Q7: What is an N+1 query problem, and how did you prevent it?

An N+1 problem occurs when fetching N parent rows triggers N additional queries for child relationships (e.g. AuditLog). We prevented this by using explicit JOIN FETCH queries in Spring Data JPA and avoiding eager fetching on collection fields in favor of indexed pagination.

Q8: What happens if the Python ML microservice crashes?

The HTTP call in TransactionService is wrapped in a resilient try-catch. If the ML microservice is unreachable or times out, the system triggers a SERVICE_DOWN system alert and gracefully falls back to the local RuleEngine to score the transaction heuristically, ensuring zero downtime for payment gateways.

Q9: What was the Next.js hydration error you encountered?

The navigation bar displayed a live clock. The server pre-rendered the page at timestamp T1, but the browser hydrated it at T2. React detected the text mismatch and threw a hydration error. I resolved it using a mounted state hook and suppressHydrationWarning, ensuring static SSR output on initial load and dynamic ticking post-mount.

Q10: How would you scale FraudShield to 50,000 transactions per second?

1) Place an API gateway (e.g., Kong) in front and publish transactions directly to partitioned Kafka topics; 2) Scale stateless Spring Boot worker pods horizontally in Kubernetes; 3) Export XGBoost models to ONNX runtime / C++ Triton server for GPU-accelerated 5ms inference; 4) Shard PostgreSQL by user_id or migrate to CockroachDB.

'What Was the Hardest Bug You Fixed?' (Interview Stories):

Scenario 1: The Silent Account Lockout (Lombok @Builder.Default)

"While testing user provisioning, newly created analysts couldn't log in despite correct credentials. In PostgreSQL, I found `is_active` was saved as false despite having private boolean `isActive = true` in Java. I discovered that Lombok's `@Builder` bypasses inline field initializers unless `@Builder.Default` is specified. I fixed this across all entities and added automated tests in `AdminServiceTest` to prevent regression."

Scenario 2: Ephemeral Socket Starvation Under Load

"During high-throughput testing, Spring Boot threw connection timeouts to the ML service after ~2,000 requests. Running `netstat` revealed thousands of sockets stuck in `TIME_WAIT`. The code was instantiating a new `RestTemplate` per request. I replaced it with a singleton `RestTemplate` backed by Apache `PoolingHttpClientConnectionManager` with strict connection pooling and timeouts."